

a. Analysis:

Naive Solution:

The outer loop runs n times and the inner loop runs about $n/2$ times, making the overall time complexity $O(n^2)$. Only a few integer variables are used, the space complexity is $O(1)$.

Time: $\Omega(1)$, $\Theta(n^2)$, $O(n^2)$

Space: $O(1)$

Optimized Solution:

Each element is processed once while lookups in the hash map are constant on average, so time complexity is $O(n)$. The hash map stores at most n elements, so the space complexity is $O(n)$.

Time: $\Omega(1)$, $\Theta(n)$, $O(n)$

Space: $O(n)$

b. Invariant & correctness

Naive Solution:

Initialization: before the first iteration, no pairs have been checked.

Maintenance: Each inner loop tests a new pair (i, j) not yet examined.

Termination: After all iterations, every pair has been tested; if none match the target, return null.

Optimized Solution:

Initialization: The hash map is empty before the loop begins.

Maintenance: For each element, check if its complement exists in the map; if not, store the element.

Termination: After all elements are processed, if no pair is found, return null

c. Output

nums = [2, 7, 11, 15], target = 9

Both programs get the same result:

Indices: 0, 1

d. Use Case

The naive approach works fine for small inputs or low-memory systems.

The optimized version is better for bigger datasets since it cuts time from $O(n^2)$ to $O(n)$.

- e. I wouldn't fully rely on LLMs for coding tasks. They're helpful for ideas but can miss logic or edge cases, so human review is still needed.